# Final Report:
# Pneumonia Detection In X-RAY images

**Swesan Pathmanathan, Muneeb Hassan, Mithun Paramathasan**

## 1 Introduction

Our project titled, Chest X-Ray Images (Pneumonia), aims to develop an automated system that predicts whether a patient is affected by pneumonia based on chest X-ray images. This task is significant because pneumonia remains one of the leading causes of death among children worldwide, and early, accurate diagnosis is critical for effective treatment. However, manual interpretation of X-rays is time-consuming and subject to human error, especially in regions with limited access to radiologists. By leveraging machine learning, our project seeks to improve diagnostic efficiency and accessibility, contributing to faster and more consistent detection. The problem is challenging due to several factors: chest X-rays often exhibit subtle differences between healthy and infected lungs, the dataset suffers from class imbalance, and variations in image quality, lighting, and patient anatomy can affect model performance. Our work will involve careful preprocessing, model selection, and evaluation to address these issues. We will also explore existing high-performing deep learning architectures and investigate techniques to enhance their accuracy, aiming to surpass the 90% benchmark reported in prior studies.

## 2 Related Work

Most of the prior research on pneumonia detection, or image recognition in general, has focused on deep learning methods such as convolutional neural networks (CNNs) and transfer learning, rather than using feature extraction approaches to predict labels. The models we decided to implement for this stage of the project differ from the techniques commonly used to tackle this problem, as we first would like to implement traditional machine learning algorithms before moving onto CNNs or pretrained models.

One of the most notable works for this problem is the CheXNet study by Rajpurkar et al. (Rajpurkar et al., 2017), which achieved radiologist level pneumonia detection by training a DenseNet based classifier. In another study, Kermany et al. (Kermany et al., 2018) demonstrated that CNNs can be improvised and transfer learning can be applied to improve the accuracy of the model given a limited dataset. Their model could effectively identify pneumonia from chest X-ray images with performance comparable to human experts.

Further studies (Hasan et al., 2021) have also explored the use of existing deep learning and pretrained models to boost accuracy. For example, Hasan et al. (Hasan et al., 2023), introduced an ensemble of pre-trained CNNs for pneumonia detection, significantly improving overall performance. Alowais et al. (Alowais et al., 2023) highlighted the benefit of applying these techniques and machine learning models in healthcare by explaining how they are helping to reduce diagnostic error and assist clinicians with early disease detection.

Overall, current findings indicate that deep learning machine learning methods tend to be used for this problem, however there is still room for exploration of simpler models and even improving accuracy of more complex models. In our project, we aim to build on these findings by comparing traditional ML baselines with CNN and transfer learning methods to understand the trade-offs and practicality of different solutions.

## 3 Dataset

The dataset consisted of 3 folders of train, val and testing. Each of these folders consisted of 2 sub-folders: Pneumonia, and no pneumonia. The dataset was downloaded from a Kaggle Pneumonia dataset.

**Pneumonia folder** This was a sub folder in all 3 of the folders (train, val, test). This subfolder consisted of chest X-ray images of patients who

were diagnosed with pneumonia.

**No Pneumonia folder** This was also a sub folder in all 3 of the folders (train, val, test). This subfolder consisted of chest X-ray images of patients who were not diagnosed with pneumonia.

**Properties of the data** The dataset was entirely image files. The data points in the dataset consisted of the image, and the y-label, which we are required to predict as either **pneumonia (1)** or **No pneumonia (0)**. This is a classical CNN problem. However, for this part of the project, we wanted to examine how traditional ML algorithms can be used to classify images, and to do that, we needed to perform feature extraction by extracting different features from each image into a tabular form. Moving forward, for the next part, we will definitely be looking at how CNNs are used to classify such images.

**Important note:** Please note that the model was not trained on the original train data folder. The original train test split had only 16 images in the validation dataset and was not enough to validate. Therefore, we split the dataset in a 60-20-20 rule. Therefore, make sure to run all the cells in the data preprocessing section of the ipynb file to get the best results.

**Preprocessing** Before we extracted features from the images, it was vital that we preprocessed the data. Firstly, we re-sized every image to a 224x224 size so we do not end up with images of different size. Next, we converted these images to tensors to be used by PyTorch. Finally, we scaled the dataset by using the mean and std dev. of the ImageNet dataset. For the train dataset, we performed data augmentation to create some diversity in the train data. After the preprocessing was complete, we moved onto feature extraction.

## 4 Features

We had to perform feature engineering on our model. The goal was to extract features from our dataset to turn our image based dataset to a more tabular dataset, so we could use traditional ML models upon it.

**Load the dataset** We loaded the train, test and val dataset in a pandas data frame before extracting features.

**Pre-trained model** We used the ResNet18 pre-trained model, which is a feature extractor trained on the enormous ImageNet dataset.

**Extract features** We then used this feature extractor to extract features from the images in our train, val, and test dataset, and store them into a tabular form in a numpy array. Now we have successfully converted our dataset consisting of images into a tabular format consisting of 512 different rows. What each row does, is none of our concern for now, since the ResNet18 model extracted them based on its pre-trained model. Our only concern here, is to work with these 512 features and train a ML model upon it. My assumption is that this method will not out perform the CNN model, however it will give us an excellent idea on how feature engineering is done using image data. Once we understand the problem better, we plan to train a CNN on our dataset moving forward.

## 5 Implementation

Before training a CNN, we implemented and compared four supervised learning models—Majority-Class Baseline, Logistic Regression, Support Vector Machine (SVM), and a Feedforward Neural Network (FNN)—to classify chest X-ray images as either pneumonia or no pneumonia. All experiments were conducted using `scikit-learn` for the traditional machine-learning models and `PyTorch` for the neural-network model. The primary goal of this phase was to establish strong baselines, identify model limitations, and understand how different architectures perform given the same feature representations.

The dataset contained a total of 5,856 samples, consisting of 5,216 training, 624 validation, and 16 testing instances. Each image was represented by a 512-dimensional feature vector derived from precomputed embeddings. The class distribution was moderately imbalanced, with 1,583 "no pneumonia" samples (27%) and 4,273 "pneumonia" samples (73%). Feature extraction required approximately five minutes on CPU hardware, and all subsequent experiments were executed on a standard laptop environment without GPU acceleration. Once we had several different models, we decided to build a **CNN** architecture. PyTorch was the primary library used to build the CNN, and we required GPU acceleration to run the training.

**Majority-Class Baseline (Dummy Classifier).** To establish a baseline performance threshold, we implemented a simple Dummy Classifier that always predicts the most frequent class in the training data. This approach provides a reference for evaluating whether the supervised models learn meaningful decision boundaries beyond class priors. The majority class identified in the training set was "no pneumonia" (class 0). The classifier achieved a validation accuracy of 37.5% and a test accuracy of 50.0%. However, both the validation and test F1 scores were 0.0, indicating the model failed to correctly identify any positive (pneumonia) samples. This result establishes the performance floor that all subsequent models were required to exceed.

**Logistic Regression.** The Logistic Regression model served as the first learning-based baseline. It was trained using an `L2` regularization penalty and optimized via stochastic gradient descent with the `lbfgs` solver. This model provided an interpretable and efficient reference point for performance evaluation, demonstrating stability and consistency across validation and test sets.

**Support Vector Machine (SVM).** The SVM model was implemented using the `scikit-learn` `SVC` class with a radial basis function (RBF) kernel. This kernel choice enabled the model to capture nonlinear relationships between features without explicit feature expansion. The regularization parameter $C$ was tuned to control the tradeoff between margin maximization and classification accuracy. Although more computationally expensive than Logistic Regression, the SVM provided a stronger separation boundary in the feature space.

**Feedforward Neural Network (FNN).** The neural model was implemented in `PyTorch` as a two-layer feedforward network with 64 hidden units and ReLU activations. It was trained for ten epochs using the Adam optimizer (lr of 0.001) and binary cross-entropy loss. The network outputs a single sigmoid neuron for binary classification. We experimented with batch sizes of 32 and 64, observing stable convergence and minimal training instability. Although this architecture is relatively shallow, it served as a first step toward testing data-driven learning on feature embeddings.

**Convolution Neural Network (CNN).** The CNN was implemented using PyTorch. The architecture consisted of several layers of convolution followed by a RELU function and Max pooling. The network outputs a binary classification. The batch size used was 64. The following simplified diagram shows the architecture that was used. Note that the actual model is alot more complex. We created the diagram to explain the model in a beginner friendly way.

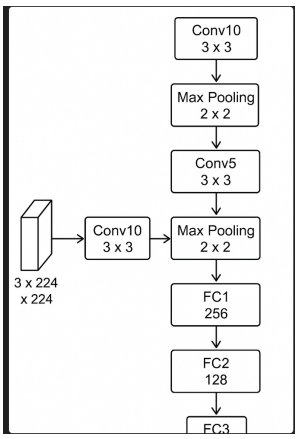


Figure 1: CNN architecture

The criterion used was cross entropy loss, and the SGD optimizer from torch.optimum was used. The learning rate was kept low 0.005 because a higher learning rate was causing overfitting. We further made use of early stopping to deduce the epoch at which the model parameter performs the best on validation set, and then use that on our test set. Please note that early stopping is not implicitly implemented within the code. We had to manually run the training at multiple different epochs along with the epoch loading bar to deduce the best epoch at which the val loss is the lowest.

## 6 Evaluation Strategy

Our evaluation strategy was designed to consistently measure each model's ability to generalize beyond the training data while reflecting the constraints of our dataset and the evolution of our methodology. In the final version of our pipeline, all images from the original train, validation, and test folders were merged into a single dataset and resplit into training (60%), validation (20%), and test (20%) subsets. This adjustment was necessary because the original validation split contained too few samples for reliable tuning, a limitation also noted during the progress-report stage. The new split ensured each partition contained sufficient representation from both pneumonia and no-pneumonia classes, enabling more stable model selection and evaluation. The training set was used

to fit model parameters, the validation set guided hyperparameter tuning and architectural choices, and the test set provided an unbiased estimate of model generalization.

Consistent with our progress-report methodology, we adopted multiple evaluation metrics to capture both overall performance and class-specific nuances, particularly important given the moderate class imbalance. We computed:

- **Accuracy** – offering a high-level view of overall correctness, as used in both the progress stage and final experiments.
- **Precision, Recall, and F1 Score** – obtained via `classification_report()` to examine minority- and majority-class behavior, mirroring our earlier emphasis on sensitivity to pneumonia cases.
- **Validation vs. Test Accuracy** – tracked across epochs to identify overfitting and ensure that improvements during training translated to unseen data.

As in the progress report, we prioritized the F1 score due to the asymmetric clinical cost of false negatives—failing to detect pneumonia represents a more serious error than a false positive. This focus allowed direct comparison between the classical models tested in the progress report and the extended set of models evaluated in the final submission.

Incorporating improvements beyond the progress-report stage, the final evaluation pipeline applied pretrained ResNet-18 feature extraction to classical models such as Logistic Regression, SVM, and a Feedforward Neural Network. These models were further strengthened using 5-fold cross-validation on the training split, providing a more robust estimate of variance than was possible earlier. In contrast, the CNN trained directly on images used a fixed validation split and epoch-level monitoring of training and validation loss, consistent with the approach introduced in the progress report but refined with data augmentation and improved preprocessing in the final stage.

Together, this expanded evaluation strategy integrates the foundational methodology introduced in the progress report—accuracy tracking, F1 emphasis, and class-specific metrics—with the more robust dataset handling and cross-validation framework implemented in the final report, providing a comprehensive and reliable basis for model comparison.

## 7 Error Analysis

To understand the strengths and weaknesses of our final CNN model, we carried out an error analysis making use of tools/approaches such as a confusion matrix, classification report and a qualitative inspection of misclassified test images. We also compared the CNN performance to all our baseline models (majority baseline, logistic regression, SVM and a feedforward neural network) to identify differences in the error patterns.

| Class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| Normal | 0.73 | 0.93 | 0.82 | 329 |
| Pneumonia | 0.97 | 0.86 | 0.91 | 843 |
| **Accuracy** | | | 0.88 | 1172 |
| **Macro Avg** | 0.85 | 0.90 | 0.86 | 1172 |
| **Weighted Avg** | 0.90 | 0.88 | 0.89 | 1172 |

Table 1: Classification report for the CNN model on the test set.

In Table 1, the metrics for the classification report are provided. The precision for the Pneumonia class is much higher than Normal, suggesting that the model is more confident in predicting Pneumonia cases. The Normal class does have higher recall however, so the model correctly identifies most Normal images, but the low precision shows that many false negatives occur (cases where predicted Normal images are actually Pneumonia).

The F1-score for Pneumonia is 0.91 beats out the F1-score for the Normal class (0.82). The support values 329 Normal vs 843 Pneumonia indicates that the dataset is significantly imbalanced, which likely contributes to the model's biased performance, as the CNN is exposed to far more pneumonia samples during training. (Valova et al., 2020) Altogether, the metrics show that the CNN does perform well overall, however it has difficulty generalizing across both classes. The lower F1-score for the Normal class combined with difference in support, aligns with the misclassification patterns highlighted later in the confusion matrix and qualitative error samples.

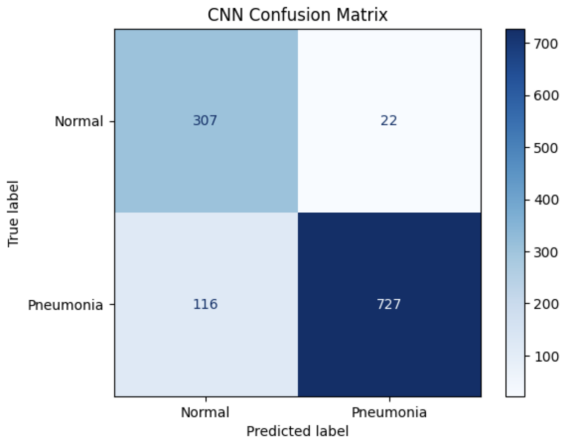


Figure 2: Confusion matrix for the CNN model on the test set. The model correctly classified 307 Normal images and 727 Pneumonia images, while misclassifying 22 Normal cases as Pneumonia and 116 Pneumonia cases as Normal.

In Figure 2, we can see that the model correctly identifies 727 Pneumonia cases, which is not bad, however the recall is low in comparison to the precision. This is not ideal as many cases of Pneumonia (116) are incorrectly predicted to be Normal cases, which is concerning in a medical context.

Our baseline models performed significantly worse than the CNN as shown in Table 2 and Figure 4/Figure 5. The CNN reached a test accuracy of 88.23% and a F1-score of 0.9133, which outperforms our best baseline model (Logistic Regression with a test accuracy of 81.25% and F1-score 0.8421). Logistic Regression finds some success in extracting feature information, however likely struggles with capturing the spatial patterns in chest X-ray images at a high accuracy. The performance difference reflects the CNN's ability to learn spatial patterns from the images, such as texture variations and regional differences in opacity, rather than relying on fixed feature extractors. This comparison demonstrates that larger deep learning models are better suited for identifying the subtle radiographic differences in these medical images and are more practical for use in medical classification contexts.

To further understand the behaviour of the CNN beyond numerical metrics, we conducted a qualitative inspection of both misclassified and correctly classified test samples as shown in Figure 3. Although we are not medically trained to interpret chest X-rays, this inspection does still provide some insight into how the model behaves.

Among the misclassified samples, the majority of errors involve pneumonia cases incorrectly predicted as normal, which aligns with our numerical metric findings. There may be various factors causing these errors such as lighting, contrast or the patient orientation. The correctly classified samples appear to have more distinct and higher contrast features. The correctly labeled images in the Figure could highlight that the model performs well when the features are more pronounced or consistent with what it learned during training. Some potential improvements that can be drawn from this inspection may be to use a deeper CNN, apply data augmentation targeted at reducing contrast/lighting variation or introducing more Normal images to help the model generalize better to challenging cases.

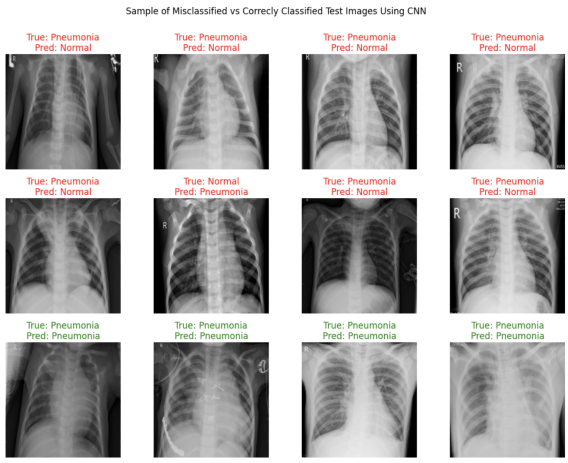


Figure 3: Sample of misclassified and correctly classified chest X-ray images from the CNN model. The top two rows show examples of misclassifications, most commonly pneumonia cases predicted as normal. The bottom row shows correctly identified pneumonia samples. These visual examples highlight the model's difficulty with subtle or low-contrast pneumonia patterns and its stronger performance on clearer, more distinct cases.

Given all our findings, we can see that our CNN excels at detecting clear cases of Pneumonia with visible features. It also effectively classifies Normal lungs with high recall and cases of Pneumonia with high precision. It leverages the advantages of improved feature extraction from adding convolutional layers allowing it to better identify spatial patterns in comparison to our baseline models. The model struggles with subtle pneumonia patterns, which often resemble Normal lungs. This introduces a critical issue in the model if it were to be applied in clinical settings. The model also appears to struggle with low contrast images where features may be obscured.

There are a couple areas for improvement we identified for the model moving forward. we can also experiment with larger CNNs making use of dropout layers to avoid overfitting we will also place these dropout layers in the convolutional part of the network rather than the fully connected part as this has been shown to potentially yield better results for chest xray imaging detection. (Szepesi and Szilágyi, 2022). Data augmentation can also be applied to reduce the sensitivity to factors such as contrast by making brightness/contrast adjustments or experimenting with cropping images in the dataset. Using a weighted cross entropy loss function or introducing more Normal samples as previously discussed could also boost the model's ability to correctly distinguish between the Normal and pneumonia samples, which is an issue due to the current imbalance in the dataset. Our implementation was also limited due to running the model on our local GPU. Using a stronger GPU would allow us to create a deeper model and further tune hyperparameters such as the learning rate, batch size and optimizer to bump up the accuracies. Discussing the misclassified images with radiologists could also be beneficial for identifying some specific patterns/features that the model may be struggling to pick up on.

| Model | Validation Accuracy | Test Accuracy | F1 Score (Test) |
|---|---|---|---|
| Majority-Class Baseline (Dummy Classifier) | 0.3750 | 0.5000 | 0.0000 |
| Logistic Regression | 0.7997 | 0.8125 | 0.8421 |
| SVM (RBF Kernel) | 0.8237 | 0.7500 | 0.8000 |
| Feedforward NN (PyTorch) | 0.7099 | 0.5625 | 0.6957 |
| **Convolutional Neural Network (CNN)** | **0.8822** | **0.8823** | **0.9133** |

Table 2: Validation and test performance for all implemented models, including the final CNN. The CNN achieves the highest test accuracy and F1 score, outperforming all baseline models.

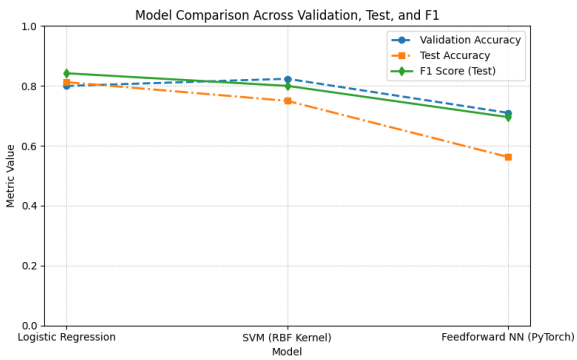


Figure 4: Model comparison across validation, test, and F1 metrics. The figure visualizes the relative performance of Logistic Regression, SVM, and the Feedforward Neural Network in alignment with the quantitative results presented in Table 2.

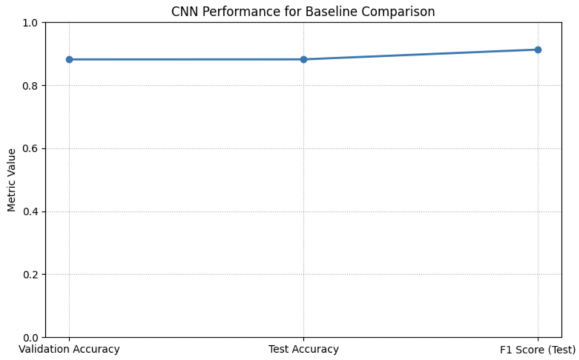


Figure 5: CNN performance across key evaluation metrics. The plot shows the validation accuracy (0.8822), test accuracy (0.8823), and F1 score (0.9133), illustrating the CNN's strong and consistent performance compared to baseline models.

## 8 Progress Reflection

In our progress report, we outlined a plan focused on transitioning from traditional machine learning models toward deeper exploration of convolutional neural networks (CNNs), incorporating data augmentation, and leveraging pre-trained architectures to improve diagnostic accuracy on chest X-ray images. Upon completing the final stage of the project, we found that we followed through on the core elements of this plan, while the specific implementation details evolved as we gained deeper insight into the dataset and the computational tradeoffs involved.

As intended, we shifted our experimentation toward end-to-end CNN models, beginning with the development of a custom baseline network trained directly on $224 \times 224$ images. This step aligned with our initial goal of comparing classical models to feature-learning architectures and allowed us to observe how spatial feature extraction improved performance relative to the purely embedding-based approach from the progress report. However, during implementation, we discovered that training deeper CNNs from scratch was less effective than anticipated due to the dataset size and class imbalance. This motivated a refinement in our plan: instead of focusing exclusively on building increasingly complex custom CNNs, we incorporated a feature-extraction pipeline using a pre-trained ResNet-18 backbone. This adjustment remained consistent with our original intent to explore state-of-the-art architectures but provided a more stable and computationally feasible approach for our dataset.

We also followed through on the progress re-

port's emphasis on data augmentation. Horizontal flips, rotations, and color jitter were implemented in the final pipeline and contributed to improved generalization during CNN training. While we initially planned to examine more advanced augmentation techniques such as synthetic oversampling or contrast-limited adaptive histogram equalization, we ultimately prioritized building a stable baseline given time and compute constraints. Although these advanced augmentations were not fully explored, the enhancements we did implement supported our models' robustness and aligned with our overarching objective.

Additionally, our plan to investigate transfer learning and pre-trained medical imaging models partially evolved. Rather than fine-tuning large models like DenseNet or VGG directly, we strategically chose to use pretrained networks only for feature extraction. This decision was influenced by practical considerations—specifically, the computational cost of fine-tuning full architectures and the limited number of images available for high-capacity models. Nevertheless, the exploration of pretrained backbones meaningfully informed our results and provided clearer insight into how deep feature representations compare to handcrafted ones.

Overall, while we adhered closely to the intentions laid out in the progress report, our plan matured as we gained a deeper understanding of our dataset and resource limitations. The direction of our final work reflects a blend of following through on our initial goals—implementing CNNs, applying augmentation, studying spatial features—and adapting our approach to ensure experimental reliability and feasibility. This combination of execution and responsive adjustment ultimately strengthened the validity of our findings and provided a more grounded comparison across classical models, feature-based models, and convolutional architectures.

## Team Contributions

| Author | Contribution |
|---|---|
| Swesan Pathmanathan | Evaluation Strategy section, Progress reflection section and Contributions section |
| Muneeb Hassan | Introduction section, Related Work section, Dataset and Features section and Model Implementation section and code. |
| Mithun Paramathasan | Error analysis section, references section, and proof-read entire document and formatting check |

Table 3: Summary of individual team member contributions.

## References


S. A. Alowais, H. Aldhyani, M. A. Khan, and 1 others. 2023. Revolutionizing healthcare: The role of artificial intelligence in clinical practice. *BMC Medical Education*, 23(1):1–15.

M. D. K. Hasan, M. A. Islam, M. T. Rahman, and M. S. Rahman. 2021. Deep learning approaches for detecting pneumonia in covid-19 patients by analyzing chest x-ray images. *Mathematical Problems in Engineering*, 2021:1–8.

R. Hasan, S. M. Azmat Ullah, A. Nandi, and A. Taher. 2023. Improving pneumonia diagnosis: A deep transfer learning cnn ensemble approach for accurate chest x-ray image analysis. In *Proceedings of the 2023 International Conference on Information and Communication Technology for Sustainable Development (ICICT4SD)*, pages 109–113.

D. S. Kermany, M. Goldbaum, W. Cai, and 1 others. 2018. Identifying medical diagnoses and treatable diseases by image-based deep learning. *Cell*, 172(5):1122–1131.e9.

Pranav Rajpurkar, Jeremy Irvin, Kaylie Zhu, Brandon Yang, Hershel Mehta, Tony Duan, Daisy Ding, Aarti Bagul, Curtis Langlotz, Katie Shpanskaya, Matthew Lungren, and Andrew Y. Ng. 2017. Chexnet: Radiologist-level pneumonia detection on chest x-rays with deep learning. *arXiv preprint arXiv:1711.05225*.

Patrik Szepesi and László Szilágyi. 2022. Detection of pneumonia using convolutional neural networks and deep learning. *Biocybernetics and Biomedical Engineering*, 42(3):1012–1022.

Iren Valova, Christopher Harris, Tony Mai, and Natacha Gueorguieva. 2020. Optimization of convolutional neural networks for imbalanced set classification. *Procedia Computer Science*, 176:660–669.